

KNN Algorithm

October 25, 2024

```
[1]: import pandas as pd
imdb_data = pd.read_csv("imdb_top_1000.csv")
imdb_data.head(3)
```

```
[1]:
```

	Poster_Link \
0	https://m.media-amazon.com/images/M/MV5BMDFkYT...
1	https://m.media-amazon.com/images/M/MV5BM2MyNj...
2	https://m.media-amazon.com/images/M/MV5BMTMxNT...

	Series_Title	Released_Year	Certificate	Runtime \
0	The Shawshank Redemption	1994	A	142 min
1	The Godfather	1972	A	175 min
2	The Dark Knight	2008	UA	152 min

	Genre	IMDB_Rating \
0	Drama	9.3
1	Crime, Drama	9.2
2	Action, Crime, Drama	9.0

	Overview	Meta_score \
0	Two imprisoned men bond over a number of years...	80.0
1	An organized crime dynasty's aging patriarch t...	100.0
2	When the menace known as the Joker wreaks havo...	84.0

	Director	Star1	Star2	Star3 \
0	Frank Darabont	Tim Robbins	Morgan Freeman	Bob Gunton
1	Francis Ford Coppola	Marlon Brando	Al Pacino	James Caan
2	Christopher Nolan	Christian Bale	Heath Ledger	Aaron Eckhart

	Star4	No_of_Votes	Gross
0	William Sadler	2343110	28,341,469
1	Diane Keaton	1620367	134,966,411
2	Michael Caine	2303232	534,858,444

```
[2]: imdb_data.isnull().sum()
```

```
[2]: Poster_Link      0
     Series_Title     0
     Released_Year    0
     Certificate      101
     Runtime          0
     Genre            0
     IMDB_Rating      0
     Overview         0
     Meta_score       157
     Director         0
     Star1            0
     Star2            0
     Star3            0
     Star4            0
     No_of_Votes      0
     Gross            169
     dtype: int64
```

```
[3]: imdb_data.shape
```

```
[3]: (1000, 16)
```

```
[4]: imdb_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 16 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Poster_Link     1000 non-null  object
1   Series_Title    1000 non-null  object
2   Released_Year   1000 non-null  object
3   Certificate      899 non-null   object
4   Runtime         1000 non-null  object
5   Genre           1000 non-null  object
6   IMDB_Rating     1000 non-null  float64
7   Overview        1000 non-null  object
8   Meta_score      843 non-null   float64
9   Director        1000 non-null  object
10  Star1           1000 non-null  object
11  Star2           1000 non-null  object
12  Star3           1000 non-null  object
13  Star4           1000 non-null  object
14  No_of_Votes     1000 non-null  int64
15  Gross           831 non-null   object
dtypes: float64(2), int64(1), object(13)
memory usage: 125.1+ KB
```

```
[5]: from sklearn.model_selection import train_test_split
      from sklearn.preprocessing import StandardScaler
      from sklearn.neighbors import KNeighborsClassifier
      from sklearn.metrics import accuracy_score
```

```
[6]: features = ['IMDB_Rating']
      X = imdb_data[features]
      y = imdb_data['Genre']
```

```
[7]: X_train, X_test, y_train, y_test = train_test_split(X, y , test_size = 0.2,
      ↪random_state = 42)
```

```
[8]: imdb_data.IMDB_Rating.value_counts()
```

```
[8]: 7.7    157
      7.8    151
      8.0    141
      8.1    127
      7.6    123
      7.9    106
      8.2     67
      8.3     44
      8.4     31
      8.5     20
      8.6     15
      8.7      5
      8.8      5
      8.9      3
      9.0      3
      9.2      1
      9.3      1
      Name: IMDB_Rating, dtype: int64
```

```
[9]: scaler = StandardScaler()
      X_train_scaled = scaler.fit_transform(X_train)
      X_test_scaled = scaler.transform(X_test)
```

```
[10]: k = 550
      knn_classifier = KNeighborsClassifier(n_neighbors = k)
      knn_classifier.fit(X_train_scaled, y_train)
```

```
[10]: KNeighborsClassifier(n_neighbors=550)
```

```
[11]: y_pred = knn_classifier.predict(X_test_scaled)
      y_pred[:10]
```

/opt/conda/envs/anaconda-panel-2023.05-py310/lib/python3.11/site-

```
packages/joblib/externals/loky/backend/context.py:110: UserWarning: Could not
find the number of physical cores for the following reason:
found 0 physical cores < 1
```

```
Returning the number of logical cores instead. You can silence this warning by
setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.
```

```
warnings.warn(
    File "/opt/conda/envs/anaconda-panel-2023.05-py310/lib/python3.11/site-
packages/joblib/externals/loky/backend/context.py", line 217, in
_count_physical_cores
    raise ValueError(
```

```
[11]: array(['Drama', 'Drama', 'Drama', 'Drama', 'Drama', 'Drama', 'Drama',
'Drama', 'Drama', 'Drama'], dtype=object)
```

```
[12]: accuracy = accuracy_score(y_test, y_pred)
print(accuracy)
```

```
0.075
```

```
[13]: # new movie
new_movie_rating = 4
new_movie = pd.DataFrame({'IMDB_Rating': [new_movie_rating]})
new_movie_scaled = scaler.transform(new_movie)
```

```
[14]: predicted_genre = knn_classifier.predict(new_movie_scaled)
print(predicted_genre)
```

```
['Drama']
```

```
[15]: from sklearn.impute import KNNImputer
imputer = KNNImputer(n_neighbors = 200)
```

```
[16]: imdb_data.Gross.value_counts()
```

```
[16]: 4,360,000      5
      25,000,000    2
      5,450,000    2
      5,321,508    2
      9,600,000    2
      ..
      106,662      1
      10,950       1
      4,018,695    1
      141,319,928  1
      30,500,000   1
      Name: Gross, Length: 823, dtype: int64
```

```
[17]: imdb_data['Gross'] = imdb_data['Gross'].str.replace(',', '')
imdb_data['Gross'] = pd.to_numeric(imdb_data['Gross'])

[18]: imdb_data_imputed = pd.DataFrame(imputer.fit_transform(imdb_data[['Gross']]),
↳ columns=['Gross_Imputed'])

[19]: imdb_data_imputed['Gross'] = imdb_data['Gross']

[20]: imdb_data_imputed.head()

[20]:   Gross_Imputed      Gross
0    28341469.0    28341469.0
1    134966411.0   134966411.0
2    534858444.0   534858444.0
3     57300000.0    57300000.0
4     4360000.0     4360000.0

[21]: imdb_data_imputed.isnull().sum()

[21]: Gross_Imputed      0
Gross              169
dtype: int64
```

0.1 You can use KNeighborsRegressor (from sklearn.neighbors) instead of using the KNNImputer for imputing missing values.

```
[51]: from sklearn.neighbors import KNeighborsRegressor
imdb_data = pd.DataFrame(imdb_data)

[55]: # Separate rows with missing values and complete rows
missing_mask = imdb_data['Gross'].isnull()

# Step 1: Training the KNN Regressor on the complete data (where 'Gross' is not
↳ missing)
X_train = imdb_data.loc[~missing_mask, ['IMDB_Rating']] # Use 'Rating' as a
↳ feature to predict 'Gross'
y_train = imdb_data.loc[~missing_mask, 'Gross'] # 'Gross' is the target

[61]: # Step 2: Fit KNN Regressor
knn_regressor = KNeighborsRegressor(n_neighbors=2) # You can tune the number
↳ of neighbors
knn_regressor.fit(X_train, y_train)

# Step 3: Predict missing 'Gross' value using the trained model
X_test = imdb_data.loc[missing_mask, ['IMDB_Rating']] # The 'Rating' for the
↳ row with missing 'Gross'
```

```

predicted_gross = knn_regressor.predict(X_test)

# Step 4: Fill in the missing value
imdb_data.loc[missing_mask, 'Gross'] = predicted_gross

imdb_data.head()

```

[61]:

```

                                Poster_Link \
0  https://m.media-amazon.com/images/M/MV5BMDFkYT...
1  https://m.media-amazon.com/images/M/MV5BM2MyNj...
2  https://m.media-amazon.com/images/M/MV5BMTMxNT...
3  https://m.media-amazon.com/images/M/MV5BMWMwMG...
4  https://m.media-amazon.com/images/M/MV5BMWU4N2...

      Series_Title  Released_Year  Certificate  Runtime \
0  The Shawshank Redemption      1994          A   142 min
1          The Godfather      1972          A   175 min
2      The Dark Knight      2008         UA   152 min
3  The Godfather: Part II      1974          A   202 min
4      12 Angry Men      1957          U    96 min

      Genre  IMDB_Rating \
0      Drama          9.3
1  Crime, Drama          9.2
2  Action, Crime, Drama    9.0
3  Crime, Drama          9.0
4  Crime, Drama          9.0

      Overview  Meta_score \
0  Two imprisoned men bond over a number of years...    80.0
1  An organized crime dynasty's aging patriarch t...   100.0
2  When the menace known as the Joker wreaks havo...    84.0
3  The early life and career of Vito Corleone in ...    90.0
4  A jury holdout attempts to prevent a miscarria...    96.0

      Director      Star1      Star2      Star3 \
0  Frank Darabont  Tim Robbins  Morgan Freeman  Bob Gunton
1  Francis Ford Coppola  Marlon Brando  Al Pacino  James Caan
2  Christopher Nolan  Christian Bale  Heath Ledger  Aaron Eckhart
3  Francis Ford Coppola  Al Pacino  Robert De Niro  Robert Duvall
4  Sidney Lumet  Henry Fonda  Lee J. Cobb  Martin Balsam

      Star4  No_of_Votes      Gross
0  William Sadler    2343110  28341469.0
1  Diane Keaton    1620367  134966411.0
2  Michael Caine    2303232  534858444.0
3  Diane Keaton    1129952  57300000.0

```

```
4      John Fiedler      689845      4360000.0
```

```
[67]: imdb_data.Gross.value_counts()
```

```
[67]: 267289695.5      29
      2500538.5       28
      4842488.0       28
      18076173.0      21
      151534373.0     17
      ..
      106662.0        1
      10950.0         1
      4018695.0       1
      141319928.0     1
      30500000.0      1
      Name: Gross, Length: 834, dtype: int64
```

```
[69]: imdb_data.Gross.isnull().sum()
```

```
[69]: 0
```

```
[ ]:
```